CHAPTER 10

Multiple Linear Regression

Intermediate

Moving from one feature to many — the same idea, more dimensions.

FROM ONE FEATURE TO MANY

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + b_px_p$$

WORKED INTERPRETATION

House Price = Base Price + (area contribution) + (bedroom contribution) + (location contribution)

If the coefficient on "extra bedroom" is +₹3,50,000, that means: **holding area, age, and location constant**, one additional bedroom is associated with a ₹3,50,000 higher predicted price.

WHY "HOLDING OTHER VARIABLES CONSTANT" MATTERS SO MUCH

Without this phrase, coefficients are easy to misread. A bedroom coefficient doesn't mean "any house with one more bedroom is worth ₹3,50,000 more" in general — bigger houses tend to have more bedrooms *and* more area, so without controlling for area, you'd be partly measuring the effect of size, not bedrooms.

Matrix Form

Advanced

The compact notation used in papers, libraries, and derivations — explained without assuming you know linear algebra.

MATRIX REPRESENTATION

$$\hat{y} = X\beta$$

`X` is a table of all feature values (rows = observations, columns = features, plus a column of 1s for the intercept). `β` is the column of coefficients (b_0, b_1, b_2, ...). Multiplying them produces the vector of predictions `ŷ` in one shot, for every row at once.

THE NORMAL EQUATION

$$\beta = (X^T X)^{-1} X^T y$$

At a high level: this equation directly solves for the coefficients that minimize SSE, in one closed-form calculation, without any iteration. `Xᵀ` is the transpose of X; the inverse "undoes" a matrix multiplication, similar to how division undoes multiplication for ordinary numbers.

DO I NEED TO CALCULATE THIS MANUALLY IN REAL ML PROJECTS?

Usually no — `scikit-learn` and every serious library handle this internally. But understanding what the equation is *doing* (finding the coefficients that minimize squared error, directly) helps you reason about failure modes like a singular `XᵀX` matrix (see multicollinearity, Chapter 18).

Train/Test Split

Beginner

Why evaluating a model on the data it trained on is a trap.

COMMON MISTAKE

A model can memorize quirks of the exact data it saw and score deceptively well on that same data, while performing much worse on new, unseen data. That gap is exactly what train/test splitting is designed to expose.

Interactive — Train / Test Split

24 training rows, 6 test rows (20%)

Purple dots train the model; teal dots are held back purely to check how well it generalizes.

Term	Meaning
Training set	Data used to fit the model's coefficients
Test set	Data held back, used only at the end to evaluate performance
Validation set	An optional third split used for tuning decisions (e.g. choosing regularization strength) before the final test-set evaluation

WHAT IS DATA LEAKAGE?

Data leakage happens when information from the test set (directly or indirectly) influences training — for example, scaling the entire dataset before splitting it, so statistics from the test set leak into the training process. The model then looks better than it really is.

An 80/20 split is common, but there's no universal magic ratio — smaller datasets sometimes need a larger test share to get a stable estimate; huge datasets can afford a small test percentage while still holding back plenty of rows.